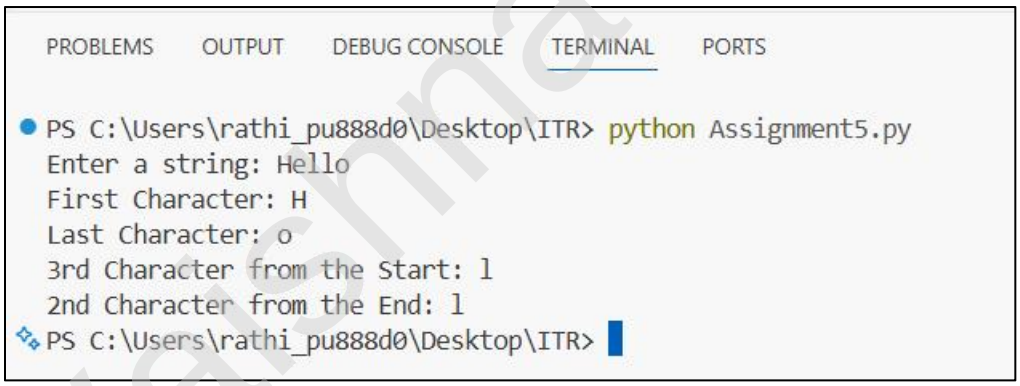
Session 05 (AIML) - Assignment Questions

Name: **Vaishnavi Praveen Rathi**
College: **Zeal Polytechnic, Narhe, Pune**
Branch: **Artificial Intelligence and Machine Learning (AIML)**
Domain: **AIML (Artificial Intelligence and Machine Learning)**

Q1. String Indexing

Output Screenshot:



Program Code:

```
""" ASSIGNMENT 5 """

'''
Q1. (String Indexing)

Write a program that takes a string input from the user.

Print:
The first character
The last character (using negative index)
The 3rd character from the start
The 2nd character from the end

Add comments explaining positive and negative indexing.
'''

# Take a string input from the user
text = input("Enter a string: ")

# Print required characters using indexing
print("First Character:", text[0])
print("Last Character:", text[-1])
print("3rd Character from the Start:", text[2])
print("2nd Character from the End:", text[-2])

'''Explanation in Comments'''

# Positive indexing starts from 0 and moves from left to right.
# Example: text[0] gives the first character, text[2] gives the third character.

# Negative indexing starts from -1 and moves from right to left.
# Example: text[-1] gives the last character, text[-2] gives the second last character.

# Indexing allows us to access individual characters of a string.
```

Q2. String Slicing

Output Screenshot:

Program Code:

```
""" ASSIGNMENT 5 """

'''
Q2. (String Slicing)

Take a string input from the user.

Using slicing, print:
a) The first 5 characters
b) The last 4 characters
c) The string in reverse order
d) Every 2nd character of the string

Include comments explaining the slicing syntax [start:end:step].
'''

# Take a string input from the user
text = input("Enter a string: ")

# Print required slices
print("First 5 Characters:", text[:5])
print("Last 4 Characters:", text[-4:])
print("String in Reverse Order:", text[::-1])
print("Every 2nd Character:", text[::2])
```

'''Explanation in Comments'''

String slicing uses the syntax [start:end:step].

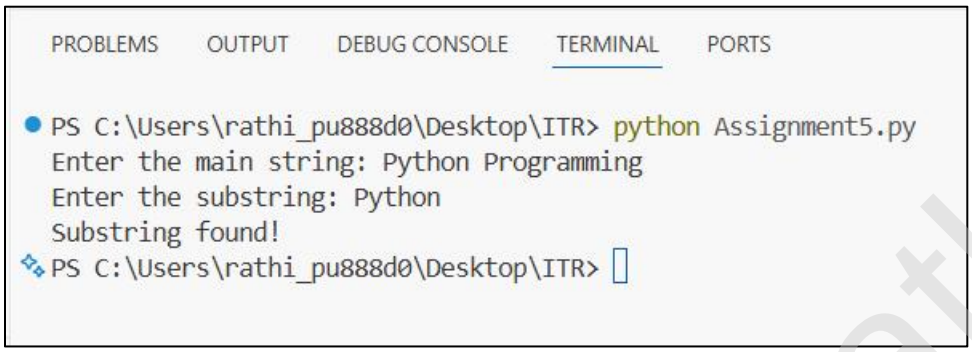
start -> Starting index (inclusive).
end -> Ending index (exclusive).
step -> Number of positions to move.

text[:5] returns characters from index 0 to 4.
text[-4:] returns the last 4 characters of the string.
text[::-1] reverses the string by moving one step backward.
text[::2] returns every second character from the string.

Slicing allows us to extract parts of a string easily.

Q3. String Membership

Output Screenshot:



Program Code:

```
""" ASSIGNMENT 5 """

'''
Q3. (String Membership)

Write a program that:

Takes a main string and a substring as input from the user.
Checks whether the substring is present in the main string using in and not in.
Prints appropriate messages (e.g., "Substring found!" or "Substring not found!").

Note that it is case-sensitive.
'''

# Take input from the user
main_string = input("Enter the main string: ")
substring = input("Enter the substring: ")

# Check whether the substring is present
if substring in main_string:
    print("Substring found!")

# Check whether the substring is not present
elif substring not in main_string:
    print("Substring not found!")
```

'''Explanation in Comments'''

The 'in' operator checks whether a substring exists in a string.
The 'not in' operator checks whether a substring does not exist in a string.
String membership checking is case-sensitive.
For example, "python" and "Python" are treated as different strings.
The program displays an appropriate message based on the result.

Q4. len() function

Output Screenshot:

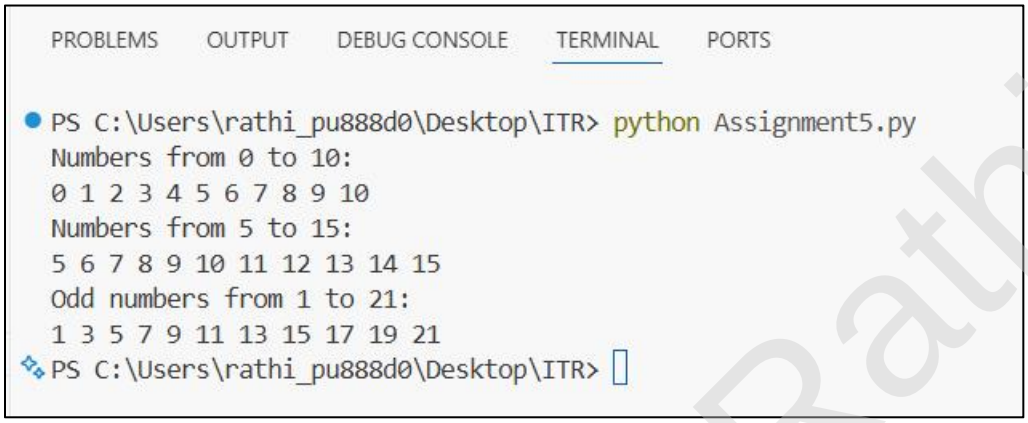
Program Code:

```
""" ASSIGNMENT 5 """

'''
Q4. (len() Function)

Write a program that:

Takes a string as input.
Prints the length of the string using len().
Prints the last character using len() (without using negative index).
Prints the middle character if the length is odd.

Add comments on common mistakes with len().
'''

# Take a string as input from the user
text = input("Enter a string: ")

# Find the length of the string
length = len(text)

# Print the length
print("Length of the string:", length)

# Print the last character using len()
print("Last Character:", text[length - 1])
```

```
# Print the middle character if the length is odd
if length % 2 != 0:
    middle_index = length // 2
    print("Middle Character:", text[middle_index])

'''Explanation in Comments'''

# len() returns the total number of characters in a string.
# Since indexing starts from 0, the last character is at position len(string) - 1.
# If the length is odd, the middle character can be found using length // 2.
```

Q5. range() - Basic Forms

Output Screenshot:



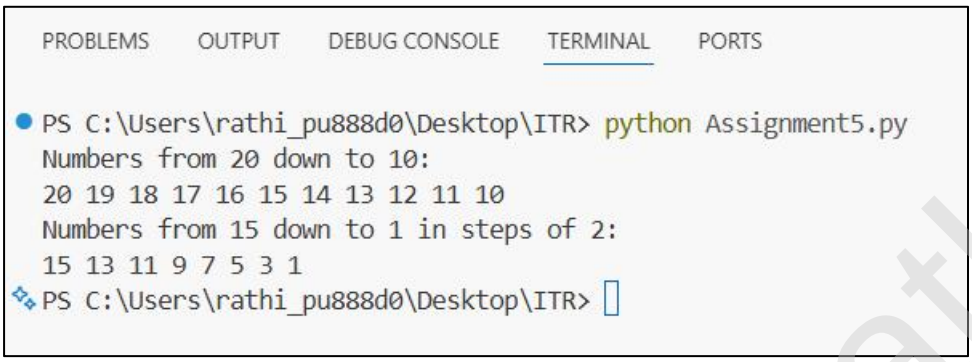
Program Code:

```
""" ASSIGNMENT 5 """

'''
Q5. (range() - Basic Forms)

Write a program using for loop and range() to:

a) Print numbers from 0 to 10.
b) Print numbers from 5 to 15.
c) Print odd numbers from 1 to 21.

Explain the three forms of range() in comments.
'''

# a) Print numbers from 0 to 10
print("Numbers from 0 to 10:")
for i in range(11):
    print(i, end=' ')

print()

# b) Print numbers from 5 to 15
print("Numbers from 5 to 15:")
for i in range(5, 16):
    print(i, end=' ')

print()
```

```python
# c) Print odd numbers from 1 to 21
print("Odd numbers from 1 to 21:")
for i in range(1, 22, 2):
    print(i, end=' ')

'''Explanation in Comments'''

# range(stop)
# Generates numbers from 0 up to (stop - 1).
# Example: range(11) generates 0 to 10.

# range(start, stop)
# Generates numbers from start up to (stop - 1).
# Example: range(5, 16) generates 5 to 15.

# range(start, stop, step)
# Generates numbers from start up to (stop - 1),
# increasing by the given step value.
# Example: range(1, 22, 2) generates odd numbers from 1 to 21.

# range() is commonly used with for loops to repeat a task a fixed number of times.
```

Q6. range() - with Negative Step

Output Screenshot:



Program Code:

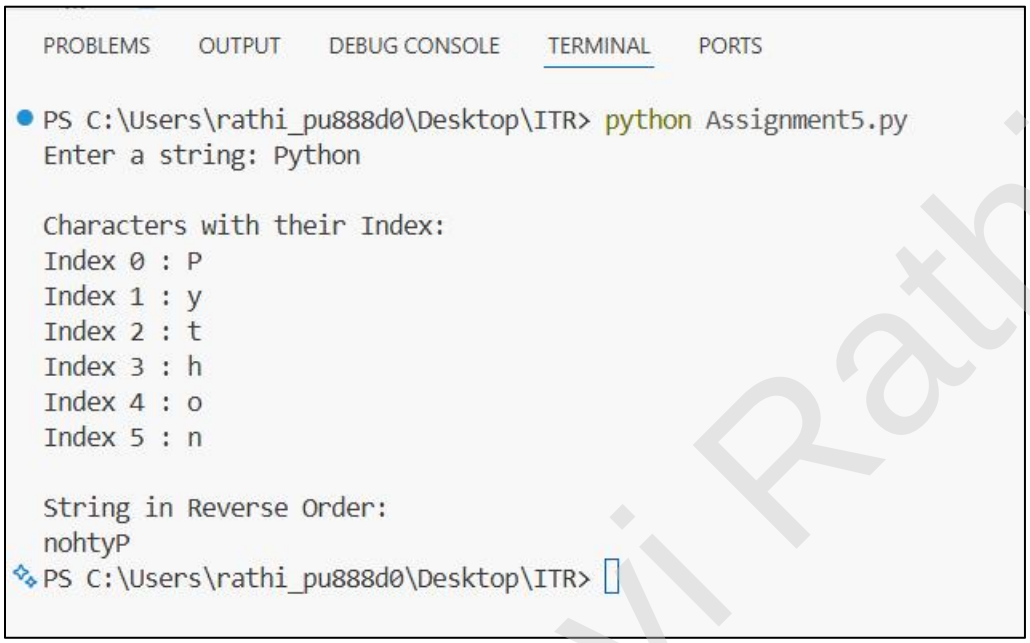
```
""" ASSIGNMENT 5 """

'''
Q6. (range() with Negative Step)

Use range() to print:

Numbers from 20 down to 10 (decreasing).
Numbers from 15 down to 1 in steps of 2.

Print each sequence on a new line with proper messages.
'''

# Print numbers from 20 down to 10
print("Numbers from 20 down to 10:")

for i in range(20, 9, -1):
    print(i, end=' ')

print()

# Print numbers from 15 down to 1 in steps of 2
print("Numbers from 15 down to 1 in steps of 2:")

for i in range(15, 0, -2):
    print(i, end=' ')
```

'''Explanation in Comments'''

A negative step value is used when counting backwards.
range(start, stop, -1) decreases the value by 1 in each iteration.
range(20, 9, -1) generates numbers from 20 to 10.
range(15, 0, -2) generates numbers from 15 to 1 with a step of 2.
The stop value is not included in the output.

Q7. Combined - Strings + range()

Output Screenshot:



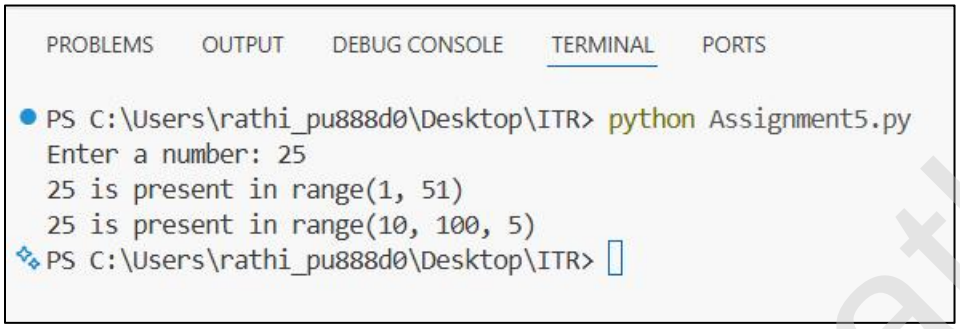
Program Code:

```
""" ASSIGNMENT 5 """

'''
Q7. (Combined: Strings + range())

Write a program that takes a string from the user.

Using a for loop and range() with len(), print:

1. Each character of the string one by one (with its index).
2. The string in reverse order using negative step in range().
'''

# Take a string input from the user
text = input("Enter a string: ")

# Print each character with its index
print("\nCharacters with their Index:")

for i in range(len(text)):
    print("Index", i, ":", text[i])
```

```
# Print the string in reverse order
print("\nString in Reverse Order:")

for i in range(len(text) - 1, -1, -1):
    print(text[i], end="")

print()

'''Explanation in Comments'''

# len(text) returns the total number of characters in the string.
# range(len(text)) generates indices from 0 to length - 1.
# These indices are used to access each character of the string.

# range(len(text) - 1, -1, -1) starts from the last index.
# The step value -1 moves backwards through the string.
# This helps print the string in reverse order.

# Combining strings, len(), and range() allows us to access characters using their positions.
```

Q8. Membership with range()

Output Screenshot:



Program Code:

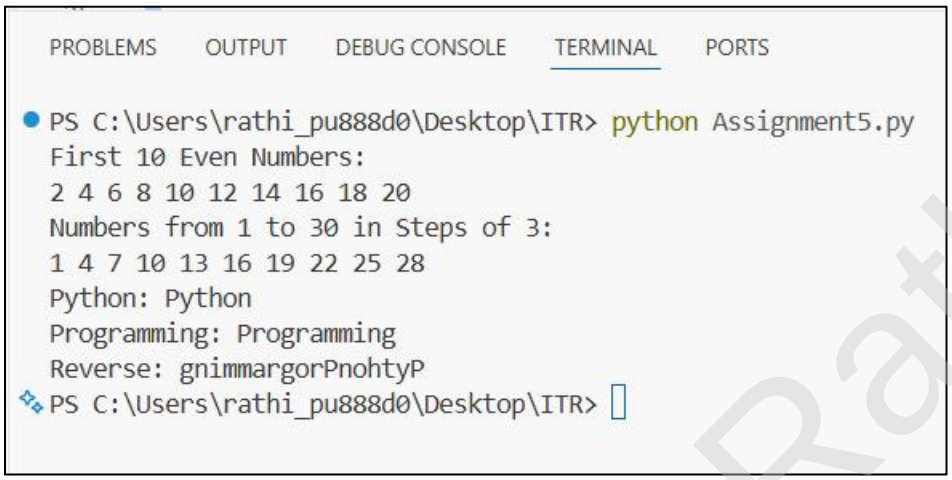
```
""" ASSIGNMENT 5 """

'''
Q8. (Membership with range())

Write a program that:

Takes a number as input from the user.
Checks if the number is present in range(1, 51) and in range(10, 100, 5).

Prints clear messages for each check.
'''

# Take a number as input from the user
num = int(input("Enter a number: "))

# Check if the number is present in range(1, 51)
if num in range(1, 51):
    print(num, "is present in range(1, 51)")
else:
    print(num, "is not present in range(1, 51)")

# Check if the number is present in range(10, 100, 5)
if num in range(10, 100, 5):
    print(num, "is present in range(10, 100, 5)")
else:
    print(num, "is not present in range(10, 100, 5)")
```

'''Explanation in Comments'''

The 'in' operator is used to check membership in a range.
range(1, 51) contains numbers from 1 to 50.
range(10, 100, 5) contains numbers starting from 10,
increasing by 5, up to 95.
If the number exists in the range, a message is displayed.
Otherwise, the program indicates that the number is not present.

Q9. Slicing + range()

Output Screenshot:



Program Code:

```
""" ASSIGNMENT 5 """

'''
Q9. (Slicing + range())

Write a program that prints the following using range() and string slicing concepts
where applicable:

1. First 10 even numbers (2 to 20).
2. Numbers from 1 to 30 in steps of 3.
3. A string "PythonProgramming" sliced to show "Python",
   "Programming", and the reverse.
'''

# Print the first 10 even numbers
print("First 10 Even Numbers:")

for i in range(2, 21, 2):
    print(i, end=' ')

print()

# Print numbers from 1 to 30 in steps of 3
print("Numbers from 1 to 30 in Steps of 3:")
```

```
for i in range(1, 31, 3):
    print(i, end=' ')

print()

# String for slicing
text = "PythonProgramming"

# Print sliced strings
print("Python:", text[:6])
print("Programming:", text[6:])
print("Reverse:", text[::-1])

'''Explanation in Comments'''

# range(2, 21, 2) generates even numbers from 2 to 20.
# range(1, 31, 3) generates numbers from 1 to 30 with a step of 3.

# text[:6] extracts characters from index 0 to 5 ("Python").
# text[6:] extracts characters from index 6 to the end ("Programming").
# text[::-1] reverses the string using slicing.

# range() is useful for generating sequences of numbers.
# Slicing is useful for extracting specific parts of a string.
```

Q10. Mini Project - Combined

Output Screenshot:

Program Code:

```
""" ASSIGNMENT 5 """

'''
Q10. (Mini Project - Combined)

Create a String Analyzer program:

Take a string input from the user.

1. Print length of the string.
2. Print first half and second half using slicing.
3. Check if the word "python" (case insensitive) is present.
```

```
4. Use range() to print all characters with their positive and negative indices.
5. Print the string in reverse.

Add proper comments and use meaningful variable names.
'''

# Take a string input from the user
user_string = input("Enter a string: ")

# Print the length of the string
string_length = len(user_string)
print("Length of the String:", string_length)

# Find the middle position
middle_index = string_length // 2

# Print first half and second half
print("First Half:", user_string[:middle_index])
print("Second Half:", user_string[middle_index:])

# Check if the word "python" is present (case insensitive)
if "python" in user_string.lower():
    print("The word 'python' is present in the string.")
else:
    print("The word 'python' is not present in the string.")

# Print characters with positive and negative indices
print("\nCharacters with Positive and Negative Indices:")

for i in range(string_length):
    negative_index = i - string_length
    print("Positive Index:", i,
          "Negative Index:", negative_index,
          "Character:", user_string[i])

# Print the string in reverse
print("\nString in Reverse:")
print(user_string[::-1])

'''Explanation in Comments'''

# len() is used to find the length of the string.
# Slicing is used to divide the string into two halves.
# lower() converts the string to lowercase for case-insensitive comparison.
# range() generates indices from 0 to length - 1.
# Positive indexing starts from 0 and moves left to right.
# Negative indexing starts from -1 and moves right to left.
# user_string[::-1] reverses the string using slicing.
# Meaningful variable names improve readability and understanding.
```